

2802ICT Intelligent Systems

Assignment 2 – Programming:

Programming: Multilayer Perceptron (MLP)

Due: Monday 28th September 2026, 11:59 PM

Total Marks: 30%

Instructions:

- **Late submission:** Late submission is allowed, but a penalty applies. The penalty is defined as the reduction of the mark allocated to the assessment item by 5% of the total weighted mark for the assessment item, for each working day that the item is late. A working day is defined as Monday to Friday. Assessment items submitted more than five working days after the due date will be awarded zero marks.
- **Extension:** You can request an extension of time on one of two grounds: medical or special circumstances (e.g. special family or personal circumstances, unavoidable commitments). All requests must be made through the myGriffith portal.
- **Individual Work:** You must complete this assignment individually and submit authentic work. This should be your own work. Anyone caught plagiarising will be penalised and reported to the university.
- **Presentation:** You must present/demonstrate your work in one-on-one conversation with a teaching staff member to discuss all components of this assignment and demonstrate your understanding of the subject. The schedule for the student demonstration will be available before the conversations. Your marks in the assignment will be modified based on this one-on-one conversation. Your work will not be marked if you do not present it.

Objectives:

- Design neural networks (Regression and multilayer perceptrons).
- Implement neural networks and learning algorithms.
- Evaluate the performance of a simple learning system with several tiny neural networks on a real-world dataset.

Your code should be written in Python. You need to learn some basics of two Python packages numpy and autograd to implement the code.

Note: You are **NOT ALLOWED** to use libraries such as sklearn, pytorch, tensorflow to implement the algorithms. You are allowed to use libraries for data structures, math functions and the libraries mentioned in the task description below.

Task 1 – Classification using Neural Networks (30 marks)

In this task, you need to implement a widely adopted machine learning algorithm, Neural Networks. You will train and test three simple neural networks in different depths with the datasets provided to you and experiment with various settings of hyperparameters.

Dataset Introduction

The dataset provided to you consists of images of fashion items. The training and test datasets are provided with this assignment in the following files:

- **Training examples:** *fashion-mnist_train.csv.gz*
- **Test examples:** *fashion-mnist_test.csv.gz*

There are 60,000 training samples and 10,000 test samples. Each sample is a grayscale image, associated with a label from 10 classes. Each image is 28×28 pixels, namely, consists of 784 pixels in total. Each pixel has a grayscale value represented as an integer between 0 and 255, with 0 indicating white and 255 indicating black. The training and test data sets have 785 columns. The first column consists of the class labels (see below) and represents the type of clothing. The rest of the 784 columns contain the pixel values of the associated image. Each training and testing example is assigned to one of the following ten classes:

Class 0:	T-shirt/top	Class 5:	Sandal
Class 1:	Trouser	Class 6:	Shirt
Class 2:	Pullover	Class 7:	Sneaker
Class 3:	Dress	Class 8:	Bag
Class 4:	Coat	Class 9:	Ankle boot

To enhance your understanding of machine learning fundamentals and explore the impact of dataset size on model performance, we have created two small training datasets by randomly sampling from the original Fashion-MNIST training set for each class:

- *fashion-mnist_train_pruned_50percent.csv.gz* (containing 50% of the original data)
- *fashion-mnist_train_pruned_10percent.csv.gz* (containing 10% of the original data)

Implementation Requirements

Your task is to implement a neural network learning algorithm that creates verified network architectures:

- 1) a Softmax regression (784-10).
- 2) an MLP with a hidden layer and a sigmoid activation function (784-30-10).
- 3) an MLP with two hidden layers and activation functions, and the residual connection from the first hidden layer to the second hidden layer (784-30-30-10). Choose your activation functions.

"xxx-xxx-xxx" denotes a network architecture. For instance, "784-30-10" means that the two-layer MLP's input dimension, hidden-layer dimension, and output dimension are 784, 30, 10, respectively.

You should implement your neural networks in two main files: **nn.py** and **train.py**. In these two files, the main framework has been implemented, and you only need to insert your code to the places marked as *"raise NotImplementedError"*.

Your code should be designed to work with the provided training framework and accept command line arguments for flexible experimentation. The main training script should be run on the command line in the following manner:

```
python train.py --model mlp --ninput 784 --nhidden 30 --noutput 10 --epochs 20 --lr 0.01 --batch_size 4 --train_file fashion-mnist_train.csv.gz --test_file fashion-mnist_test.csv.gz --data_dir ./data
```

Where the meaning of each argument is:

- model: Type of neural network (linear, mlp, mlp3)
- ninput: Number of neurons in the input layer (784 for Fashion-MNIST)
- nhidden: Number of neurons in the hidden layer(s)
- noutput: Number of neurons in the output layer (10 for Fashion-MNIST)
- epochs: Number of training epochs
- lr: Learning rate
- batch_size: Mini-batch size for training
- train_file: Path to the training dataset file
- test_file: Path to the test dataset file
- data_dir: Directory containing the data files

The desired output is shown in Appendix 3.

Supplementary Information

NumPy and autograd are two main libraries that are supposed to be utilised in this Assignment. Appendix 1 and 2 introduce some basic ideas of these two packages.

The codes associated with data loading and plotting have been implemented completely. The framework uses `numpy.loadtxt()` with gzip support to read from compressed CSV files. There is no need to uncompress the files before use. For plotting, you should use the Python library `matplotlib`.

You will implement gradient descent using automatic differentiation (autograd library). The framework handles the gradient computation automatically, but you need to implement: Forward pass computations for each network type; Cross-entropy loss function; Parameter update rules for gradient descent.

It is important that your code enables simple and easy adjustment of key parameters such as the number of neurons in the input, hidden, and output layers; number of training epochs; mini-batch size; learning rate; and model architecture type. While the provided template has implemented

these functions, you need to understand how to run the code in various settings by modifying input parameters rather than overwriting the code.

Required Experiments

You should conduct the following experiments and document the results. The best way is to demonstrate results in Tables or Figures.

(a) Baseline Network Training: Under the default settings of learning epochs (20), mini-batch size (4), and learning rate (0.01). Create four different neural networks: Softmax regression, two-layer MLP, three-layer MLP without residual connections, and three-layer MLP with a residual connection. Run experiments for each network on three datasets (one full set and two pruned sets). Try to compare different neural networks on different datasets.

(b) Learning Rate Analysis: Select one specific network architecture, train the network on the full dataset with the same settings as in (a) but with different learning rates: 0.001, 0.01, 1.0, 10, 100. Which learning rate achieves the maximum test accuracy? Which ones are totally unstable for training?

(c) Mini-batch Size Study: Select one specific network architecture, train the network on the full dataset with the same settings as in (a), but with different mini-batch sizes = 1, 4, 8, 16. Which mini-batch size achieves the maximum test accuracy? Which one is the slowest to train?

Report Requirements

You should write a report for the programming task and it should contain the following components.

- Software design: Detailed description of your neural network implementations
- Description of how the forward pass, loss computation, and parameter updates work
- **Experimental Results Section:**
 - All graphs and numerical results from experiments (a) through (c); include screenshots showing code running.
 - Clear presentation of accuracy measurements and training curves
 - Comparison tables showing performance across different hyperparameter settings
- **Analysis and Discussion:**
 - Detailed explanations of the experimental results in experiments (a) and the logic behind the observed patterns
 - Analysis of how different learning rates affect convergence and final performance
 - Discussion of the relationship between mini-batch size and training efficiency/accuracy
- Write a conclusion paragraph that summarizes and explains your findings.

Submission

You should submit the following documents through the link provided in L@G by the due date:

1. A copy of your report as specified above. Name the file:

firstname_surname_Snumber_A2_report (for example, Bart_Simpson_s123456_A2_report.pdf)

2. A zip file that includes your code (.py files). Name the file:

firstname_surname_Snumber_A2_code.zip (for example, Bart_Simpson_s123456_A2_code.zip)

Note: Make sure your code is well documented with clear comments explaining the key implementation details, especially in the sections where you complete the NotImplementedError placeholders.

Marking Scheme

A detailed marking rubric is provided with this document in L@G.

***** End of Assignment Description *****

APPENDIX

Appendix 1. NumPy

NumPy (Numerical Python) is the fundamental package for scientific computing in Python. It provides a powerful N-dimensional array object and tools for working with these arrays. In neural network implementations, NumPy serves as the backbone for efficient numerical computations.

If you wanted to learn NumPy in depth, please refer to <https://numpy.org/doc/stable/user/> or any other online learning materials. We also describe some basic concepts and operations in NumPy for you to quickly get a brief idea.

```
# Core Data Structure: ndarray
import numpy as np

# Creating arrays from lists
arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([[1, 2], [3, 4]])

# Creating arrays with specific functions
zeros = np.zeros((3, 4))      # 3x4 array of zeros
ones = np.ones((2, 3))       # 2x3 array of ones
random_arr = np.random.randn(2, 3) # Random normal distribution

# Array Properties: Shape, size, and dimensions
print(arr2.shape) # (2, 2) dimensions
print(arr2.size)  # 4 - total number of elements
print(arr2.ndim)  # 2 number of dimensions
print(arr2.dtype) # data type of elements

# Matrix Operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Element-wise multiplication
elementwise = A * B

# Matrix multiplication (crucial for neural networks)
matrix_mult = np.dot(A, B) # or A @ B in Python 3.5+

# Broadcasting
# Broadcasting allows operations between arrays of different shapes
# For instance, adding bias to each sample
X = np.random.randn(5, 3) # 5 samples, 3 features
bias = np.array([1, 2, 3]) # bias for each feature
result = X + bias          # bias is broadcasted to each row

# Indexing and Slicing (like Python List but much more powerful)
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr[0, 1]) # Element at row 0, column 1
print(arr[:, 1]) # All rows, column 1
print(arr[1:, :2]) # Rows 1 onwards, first 2 columns
```

```
# Boolean indexing (useful for data preprocessing)
mask = arr > 5
filtered = arr[mask] # Elements greater than 5

# Mathematical Functions and Statistical Operations
np.exp(-X)
np.maximum(0, X)
data = np.random.randn(100, 10)
mean = np.mean(data, axis=0)
std = np.std(data, axis=0)
sum_vals = np.sum(data, axis=1)
argmax = np.argmax(data, axis=1)
```

Appendix 2. Autograd for Automatic Differentiation

Autograd is a Python library that can automatically differentiate native Python and NumPy code. It's essential for implementing gradient-based optimization algorithms like backpropagation in neural networks without manually computing gradients.

To install autograd, simply run this command: `pip install autograd`. If you wanted to learn autograd in depth, please refer to <https://github.com/HIPS/autograd> or any other online learning materials. We also describe some basic concepts and operations in autograd for you to quickly get a brief idea.

In this assignment, autograd can be understood as an intelligent wrapper around NumPy that tracks computational operations to enable automatic differentiation. When you import `autograd.numpy` as `anp`, you're using NumPy functions that have been enhanced with gradient-tracking capabilities. Each operation performed with `anp` (like `anp.dot()`, `anp.exp()`, `anp.sum()`, originally they should be `np.dot()`, `np.exp()`, `np.sum()`, respectively) not only computes the forward result but also records the operation in a computational graph. This graph remembers how each output was computed from its inputs, along with the local gradients of each operation. When you call `grad(function)`, autograd traverses this computational graph backwards (reverse-mode automatic differentiation), applying the chain rule to compute the gradient of the final output with respect to the input parameters. Essentially, autograd transforms your familiar NumPy code into a differentiable version without changing the syntax, making it seamless to compute gradients for optimization algorithms like gradient descent in neural networks.

```
# Basic Setup
import autograd.numpy as anp # Use autograd's wrapped numpy
from autograd import grad
# Note: Use anp instead of np for functions you want to differentiate

# Example 1: Simple function
def simple_function(x):
    return anp.sum(x**2)

# Create gradient function
grad_simple = grad(simple_function)

# Compute gradient
```

```
x = anp.array([1.0, 2.0, 3.0])
gradient = grad_simple(x) # Returns [2.0, 4.0, 6.0]
```

Appendix 3. Desired Output

The desired output when executing train.py with default arguments is shown below:

```
FASHION-MNIST NEURAL NETWORK TRAINING
Model: LINEAR
Input size: 784
Output size: 10
Epochs: 20
Learning rate: 0.01
Batch size: 4
Data directory: ./data
Validation ratio: 0.2
Random seed: 42

Loading Fashion-MNIST dataset...
Training data shape: (60000, 784), Labels shape: (60000,)
Test data shape: (10000, 784), Labels shape: (10000,)
Number of classes: 10

Creating validation split with 20% of training data...
Class 0: 6000 -> Train: 4800, Val: 1200
Class 1: 6000 -> Train: 4800, Val: 1200
Class 2: 6000 -> Train: 4800, Val: 1200
Class 3: 6000 -> Train: 4800, Val: 1200
Class 4: 6000 -> Train: 4800, Val: 1200
Class 5: 6000 -> Train: 4800, Val: 1200
Class 6: 6000 -> Train: 4800, Val: 1200
Class 7: 6000 -> Train: 4800, Val: 1200
Class 8: 6000 -> Train: 4800, Val: 1200
Class 9: 6000 -> Train: 4800, Val: 1200

Final split:
Training set: 48000 samples
Validation set: 12000 samples

Created Linear Classifier
Starting training for 20 epochs...
Learning rate: 0.01, Batch size: 4

Epoch 1/20 Train Loss: 1.0567 | Val Loss: 1.0077 | Test Loss: 0.9986 | Train Acc: 83.00% | Val Acc:
82.42% | Test Acc: 83.00% | Time: 4.86s
*** New best validation accuracy! ***
Epoch 5/20 Train Loss: 0.7669 | Val Loss: 0.9823 | Test Loss: 0.8831 | Train Acc: 84.76% | Val Acc:
84.09% | Test Acc: 84.71% | Time: 4.10s
*** New best validation accuracy! ***
Epoch 10/20 Train Loss: 0.7298 | Val Loss: 0.8787 | Test Loss: 0.8530 | Train Acc: 85.66% | Val Acc:
84.56% | Test Acc: 85.34% | Time: 4.63s
*** New best validation accuracy! ***
Epoch 15/20 Train Loss: 0.7168 | Val Loss: 0.8668 | Test Loss: 0.8512 | Train Acc: 86.09% | Val Acc:
84.76% | Test Acc: 85.23% | Time: 4.18s
*** New best validation accuracy! ***
Epoch 20/20 Train Loss: 0.7063 | Val Loss: 0.8653 | Test Loss: 0.8502 | Train Acc: 86.21% | Val Acc:
84.86% | Test Acc: 85.44% | Time: 4.42s
```

Model selection results:

Best validation accuracy: 84.92% (epoch 19)

Test accuracy of best model: 85.50%

TRAINING COMPLETED

Total training time: 86.69 seconds

Final training accuracy: 86.21%

Best validation accuracy: 84.92% (epoch 19)

Test accuracy of best model: 85.50%

Final training loss: 0.7063

Final validation loss: 0.8653

Final test loss: 0.8502

Training history plot saved as 'linear_classifier_training_history.png'